

**Федеральное государственное автономное образовательное учреждение
высшего образования**

"Национальный исследовательский университет

"Высшая школа экономики"

Московский институт электроники и математики им. А. Н. Тихонова

Департамент компьютерной инженерии

Отчёт по дисциплине

«Распределённые базы данных и сетевые вычисления»

Объектная модель в Java

Студент:

Абрамов Илья Александрович

Группа: МКС254

Преподаватель: Байбикова Т.Н.

1 Задание

Разработать класс Aeroplane. В разработанной программе отобразите следующие моменты:

- конструкторы и их перегрузку;
- перегрузку методов;
- наследование (двух видов – быть чем-то и агрегацию);
- виртуальные методы;
- абстрактные классы;
- полиморфизм;
- интерфейсы.

2 Разработанная программа

Всего было создано 8 файлов:

- Flyable.java - Интерфейс для летающих объектов
 - Определяет методы: takeOff(), land(), fly(), getFuelConsumption()
- Maintainable.java - Интерфейс для обслуживаемых объектов
 - Определяет методы: performMaintenance(), getMaintenanceInterval(), needsMaintenance()
- Engine.java - Класс двигателя
 - Демонстрирует: перегрузку конструкторов (4 варианта), перегрузку методов (start)
 - Используется в HAS-A relationship с Aeroplane
- Vehicle.java - Абстрактный базовый класс транспортного средства
 - Демонстрирует: абстрактные методы, виртуальные методы, перегрузку конструкторов
 - Абстрактные методы: start(), stop(), getType()
 - Виртуальный метод: displayInfo()
- Aeroplane.java - Основной класс самолета
 - Наследует Vehicle, реализует Flyable и Maintainable
 - Демонстрирует: IS-A (наследование), HAS-A (агрегация Engine), перегрузку конструкторов и методов
 - Переопределяет все абстрактные и виртуальные методы
- PassengerPlane.java - Класс пассажирского самолета
 - Наследует Aeroplane
 - Демонстрирует: многоуровневое наследование, переопределение методов, перегрузку методов
 - Специфические методы: boardPassengers()
- CargoPlane.java - Класс грузового самолета
 - Наследует Aeroplane

- Демонстрирует: многоуровневое наследование, переопределение методов, перегрузку методов
- Специфические методы: loadCargo(), unloadCargo()
- App.java
 - Главный класс с методом main
 - Полная демонстрация всех концепций ООП
 - Организован в 10 секций, каждая демонстрирует отдельную концепцию

3 Листинг

3.1 Файл Flyable.java

```
public interface Flyable {  
    void takeOff();  
    void land();  
    void fly(int altitude);  
    double getFuelConsumption();  
}
```

3.2 Файл Maintainable.java

```
public interface Maintainable {  
    void performMaintenance();  
    int getMaintenanceInterval();  
    boolean needsMaintenance();  
}
```

3.3 Файл Engine.java

```
public class Engine {
    private String model;
    private int horsepower;
    private double fuelConsumptionRate;

    // ПЕРЕГРУЗКА КОНСТРУКТОРОВ
    public Engine() {
        this("Unknown", 1000, 5.0);
    }

    public Engine(String model) {
        this(model, 1000, 5.0);
    }

    public Engine(String model, int horsepower) {
        this(model, horsepower, 5.0);
    }

    public Engine(String model, int horsepower, double fuelConsumptionRate) {
        this.model = model;
        this.horsePower = horsepower;
        this.fuelConsumptionRate = fuelConsumptionRate;
    }

    // ПЕРЕГРУЗКА МЕТОДОВ
    public void start() {
        System.out.println("Engine " + model + " starting...");
    }

    public void start(boolean quickStart) {
        if (quickStart) {
            System.out.println("Engine " + model + " quick starting!");
        } else {
            start();
        }
    }

    public void stop() {
        System.out.println("Engine " + model + " stopped.");
    }

    public String getInfo() {
        return "Engine: " + model + ", HP: " + horsepower + ", Consumption: " +
        fuelConsumptionRate + " L/h";
    }

    public double getFuelConsumptionRate() {
        return fuelConsumptionRate;
    }
}
```

```
public int getHorsePower() {  
    return horsePower;  
}  
}
```


3.4 Файл Vehicle.java

```
public abstract class Vehicle {
    protected String manufacturer;
    protected String model;
    protected int year;
    protected double maxSpeed;

    // ПЕРЕГРУЗКА КОНСТРУКТОРОВ
    public Vehicle() {
        this("Unknown", "Unknown", 2024, 0.0);
    }

    public Vehicle(String manufacturer, String model) {
        this(manufacturer, model, 2024, 0.0);
    }

    public Vehicle(String manufacturer, String model, int year, double maxSpeed)
    {
        this.manufacturer = manufacturer;
        this.model = model;
        this.year = year;
        this.maxSpeed = maxSpeed;
    }

    // ВИРТУАЛЬНЫЙ МЕТОД - может быть переопределен в подклассах
    public void displayInfo() {
        System.out.println("Vehicle: " + manufacturer + " " + model + " (" +
year + ")");
        System.out.println("Max Speed: " + maxSpeed + " km/h");
    }

    // АБСТРАКТНЫЕ МЕТОДЫ - должны быть реализованы в подклассах
    public abstract void start();
    public abstract void stop();
    public abstract String getType();

    // Геттеры
    public String getManufacturer() {
        return manufacturer;
    }
    public String getModel() {
        return model;
    }
    public int getYear() {
        return year;
    }
    public double getMaxSpeed() {
        return maxSpeed;
    }
}
```

3.5 Файл Aeroplane.java

```
public class Aeroplane extends Vehicle implements Flyable, Maintainable {
    private int passengerCapacity;
    private double wingspan;
    private Engine engine; // HAS-A RELATIONSHIP - агрегация
    private int flightHours;
    private boolean isFlying;

    // ПЕРЕГРУЗКА КОНСТРУКТОРОВ
    public Aeroplane() {
        super();
        this.passengerCapacity = 0;
        this.wingspan = 0.0;
        this.engine = new Engine();
        this.flightHours = 0;
        this.isFlying = false;
    }

    public Aeroplane(String manufacturer, String model) {
        super(manufacturer, model);
        this.passengerCapacity = 100;
        this.wingspan = 35.0;
        this.engine = new Engine();
        this.flightHours = 0;
        this.isFlying = false;
    }

    public Aeroplane(String manufacturer, String model, int year, double
maxSpeed,
                    int passengerCapacity, double wingspan) {
        super(manufacturer, model, year, maxSpeed);
        this.passengerCapacity = passengerCapacity;
        this.wingspan = wingspan;
        this.engine = new Engine();
        this.flightHours = 0;
        this.isFlying = false;
    }

    public Aeroplane(String manufacturer, String model, int year, double
maxSpeed,
                    int passengerCapacity, double wingspan, Engine engine) {
        super(manufacturer, model, year, maxSpeed);
        this.passengerCapacity = passengerCapacity;
        this.wingspan = wingspan;
        this.engine = engine;
        this.flightHours = 0;
        this.isFlying = false;
    }

    // ПЕРЕОПРЕДЕЛЕНИЕ АБСТРАКТНЫХ МЕТОДОВ
```

```

@Override
public void start() {
    System.out.println("\n=== Starting " + manufacturer + " " + model + "
===");
    engine.start();
    System.out.println("Aircraft systems initialized.");
}
@Override
public void stop() {
    if (isFlying) {
        System.out.println("Cannot stop while flying! Land first.");
        return;
    }
    System.out.println("\n=== Stopping " + manufacturer + " " + model + "
===");
    engine.stop();
    System.out.println("Aircraft systems shut down.");
}
@Override
public String getType() {
    return "Aeroplane";
}
// ПЕРЕОПРЕДЕЛЕНИЕ ВИРТУАЛЬНОГО МЕТОДА
@Override
public void displayInfo() {
    super.displayInfo();
    System.out.println("Passenger Capacity: " + passengerCapacity);
    System.out.println("Wingspan: " + wingspan + " m");
    System.out.println("Flight Hours: " + flightHours);
    System.out.println(engine.getInfo());
}
// РЕАЛИЗАЦИЯ ИНТЕРФЕЙСА Flyable
@Override
public void takeOff() {
    if (isFlying) {
        System.out.println("Already in the air!");
        return;
    }
    System.out.println("\n>>> " + model + " is taking off...");
    engine.start(true);
    System.out.println("Accelerating on runway...");
    System.out.println("Lifting off!");
    isFlying = true;
}
@Override
public void land() {
    if (!isFlying) {
        System.out.println("Already on the ground!");
        return;
    }
    System.out.println("\n<<< " + model + " is landing...");
}

```

```

        System.out.println("Descending...");
        System.out.println("Touchdown!");
        System.out.println("Taxiing to gate.");
        isFlying = false;
        flightHours += 2; // Симуляция 2 часа полета
    }

    @Override
    public void fly(int altitude) {
        if (!isFlying) {
            System.out.println("Cannot fly - not in the air! Take off first.");
            return;
        }
        System.out.println("Flying at " + altitude + " meters altitude.");
    }

    @Override
    public double getFuelConsumption() {
        return engine.getFuelConsumptionRate() * passengerCapacity / 100.0;
    }

    // РЕАЛИЗАЦИЯ ИНТЕРФЕЙСА Maintainable
    @Override
    public void performMaintenance() {
        System.out.println("\n*** Performing maintenance on " + model + " ***");
        System.out.println("Checking engine...");
        System.out.println("Inspecting wings and fuselage...");
        System.out.println("Testing avionics...");
        System.out.println("Maintenance complete!");
        flightHours = 0; // Сброс счетчика после обслуживания
    }

    @Override
    public int getMaintenanceInterval() { return 100;}
    @Override
    public boolean needsMaintenance() {
        return flightHours >= getMaintenanceInterval();
    }
    // ПЕРЕГРУЗКА МЕТОДОВ
    public void fly() {
        fly(10000); // По умолчанию 10000 метров
    }

    public void fly(int altitude, String direction) {
        fly(altitude);
        System.out.println("Heading: " + direction);
    }

    public int getPassengerCapacity() { return passengerCapacity; }
    public boolean isFlying() { return isFlying; }
    public int getFlightHours() { return flightHours; }
}

```

3.6 Файл PassengerPlane.java

```
public class PassengerPlane extends Aeroplane {
    private int businessClassSeats;
    private int economyClassSeats;
    private boolean hasWiFi;

    // ПЕРЕГРУЗКА КОНСТРУКТОРОВ
    public PassengerPlane() {
        super();
        this.businessClassSeats = 20;
        this.economyClassSeats = 180;
        this.hasWiFi = true;
    }

    public PassengerPlane(String manufacturer, String model, int year, double
maxSpeed,
                                int passengerCapacity, double wingspan, Engine engine,
                                int businessClassSeats, int economyClassSeats, boolean
hasWiFi) {
        super(manufacturer, model, year, maxSpeed, passengerCapacity, wingspan,
engine);
        this.businessClassSeats = businessClassSeats;
        this.economyClassSeats = economyClassSeats;
        this.hasWiFi = hasWiFi;
    }

    // ПЕРЕОПРЕДЕЛЕНИЕ МЕТОДА - демонстрация полиморфизма
    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Business Class Seats: " + businessClassSeats);
        System.out.println("Economy Class Seats: " + economyClassSeats);
        System.out.println("WiFi Available: " + (hasWiFi ? "Yes" : "No"));
    }

    @Override
    public String getType() {
        return "Passenger Plane";
    }

    // ПЕРЕГРУЗКА МЕТОДОВ
    public void boardPassengers() {
        System.out.println("Boarding passengers...");
        System.out.println("Business class boarding first.");
        System.out.println("Economy class now boarding.");
    }

    public void boardPassengers(int count) {
        System.out.println("Boarding " + count + " passengers...");
    }
}
```

```
public void boardPassengers(int businessCount, int economyCount) {  
    System.out.println("Boarding " + businessCount + " business and " +  
        economyCount + " economy passengers.");  
}  
  
public void serveMeal() {  
    System.out.println("Serving meals to passengers.");  
}  
}
```

3.7 Файл CargoPlane.java

```
public class CargoPlane extends Aeroplane {
    private double maxCargoWeight;
    private double cargoVolumeCapacity;
    private boolean hasLoadingRamp;

    // ПЕРЕГРУЗКА КОНСТРУКТОРОВ
    public CargoPlane() {
        super();
        this.maxCargoWeight = 50000.0;
        this.cargoVolumeCapacity = 300.0;
        this.hasLoadingRamp = true;
    }

    public CargoPlane(String manufacturer, String model, int year, double
maxSpeed,
                        double wingspan, Engine engine,
                        double maxCargoWeight, double cargoVolumeCapacity, boolean
hasLoadingRamp) {
        super(manufacturer, model, year, maxSpeed, 0, wingspan, engine);
        this.maxCargoWeight = maxCargoWeight;
        this.cargoVolumeCapacity = cargoVolumeCapacity;
        this.hasLoadingRamp = hasLoadingRamp;
    }

    // ПЕРЕОПРЕДЕЛЕНИЕ МЕТОДА - демонстрация полиморфизма
    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Max Cargo Weight: " + maxCargoWeight + " kg");
        System.out.println("Cargo Volume: " + cargoVolumeCapacity + " m³");
        System.out.println("Loading Ramp: " + (hasLoadingRamp ? "Yes" : "No"));
    }

    @Override
    public String getType() {
        return "Cargo Plane";
    }

    // ПЕРЕГРУЗКА МЕТОДОВ
    public void loadCargo() {
        System.out.println("Loading cargo...");
    }

    public void loadCargo(double weight) {
        if (weight > maxCargoWeight) {
            System.out.println("Warning: Cargo weight " + weight + " kg exceeds
maximum " + maxCargoWeight + " kg!");
        } else {
            System.out.println("Loading " + weight + " kg of cargo.");
        }
    }
}
```

```
    }  
}  
  
public void loadCargo(double weight, String cargoType) {  
    loadCargo(weight);  
    System.out.println("Cargo type: " + cargoType);  
}  
  
public void unloadCargo() {  
    System.out.println("Unloading cargo...");  
}  
  
public double getMaxCargoWeight() {  
    return maxCargoWeight;  
}  
}
```


3.8 Файл App.java

```
public class App {
    public static void main(String[] args) {
        System.out.println("=".repeat(70));
        System.out.println("ДЕМОНСТРАЦИЯ ОБЪЕКТНОЙ МОДЕЛИ В JAVA - AEROPLANE");
        System.out.println("=".repeat(70));

        // ===== 1. ПЕРЕГРУЗКА КОНСТРУКТОРОВ =====
        System.out.println("\n" + "=".repeat(70));
        System.out.println("1. ДЕМОНСТРАЦИЯ ПЕРЕГРУЗКИ КОНСТРУКТОРОВ");
        System.out.println("=".repeat(70));

        Engine engine1 = new Engine();
        Engine engine2 = new Engine("Rolls-Royce Trent 900");
        Engine engine3 = new Engine("CFM56-7B", 27300);
        Engine engine4 = new Engine("GE90-115B", 52000, 12.5);

        System.out.println("Конструктор без параметров: " + engine1.getInfo());
        System.out.println("Конструктор с 1 параметром: " + engine2.getInfo());
        System.out.println("Конструктор с 2 параметрами: " + engine3.getInfo());
        System.out.println("Конструктор с 3 параметрами: " + engine4.getInfo());

        // ===== 2. HAS-A RELATIONSHIP (Агрегация) =====
        System.out.println("\n" + "=".repeat(70));
        System.out.println("2. ДЕМОНСТРАЦИЯ HAS-A RELATIONSHIP (АГРЕГАЦИЯ)");
        System.out.println("=".repeat(70));

        Engine boeingEngine = new Engine("GE90-115B", 52000, 12.5);
        Aeroplane boeing777 = new Aeroplane("Boeing", "777-300ER", 2020, 905.0,
                                             396, 64.8, boeingEngine);

        System.out.println("✓ Самолет Boeing 777 СОДЕРЖИТ (HAS-A) двигатель
GE90-115B");

        // ===== 3. IS-A RELATIONSHIP (Наследование) =====
        System.out.println("\n" + "=".repeat(70));
        System.out.println("3. ДЕМОНСТРАЦИЯ IS-A RELATIONSHIP (НАСЛЕДОВАНИЕ)");
        System.out.println("=".repeat(70));

        Engine airbusEngine = new Engine("Rolls-Royce Trent 900", 36300, 11.8);
        PassengerPlane airbus380 = new PassengerPlane("Airbus", "A380", 2021,
945.0,
                                             853, 79.8, airbusEngine,
                                             100, 753, true);

        Engine cargoEngine = new Engine("GE CF6-80C2", 28000, 10.5);
        CargoPlane boeing747F = new CargoPlane("Boeing", "747-8F", 2019, 908.0,
68.4, cargoEngine,
140000.0, 858.0, true);

        System.out.println("✓ PassengerPlane IS-A Aeroplane IS-A Vehicle");
    }
}
```

```

System.out.println("✓ CargoPlane IS-A Aeroplane IS-A Vehicle");

// ===== 4. ПОЛИМОРФИЗМ И ВИРТУАЛЬНЫЕ МЕТОДЫ =====
System.out.println("\n" + "=".repeat(70));
System.out.println("4. ДЕМОСТРАЦИЯ ПОЛИМОРФИЗМА И ВИРТУАЛЬНЫХ
МЕТОДОВ");
System.out.println("=".repeat(70));

// Массив типа Vehicle, содержащий разные типы самолетов
Vehicle[] fleet = new Vehicle[] {
    boeing777,
    airbus380,
    boeing747F
};

System.out.println("Полиморфное поведение - вызов методов через базовый
тип:");
for (int i = 0; i < fleet.length; i++) {
    System.out.println("\n--- Объект " + (i + 1) + " ---");
    fleet[i].displayInfo(); // Виртуальный метод - вызовется
переопределенная версия
    System.out.println("Type: " + fleet[i].getType());
}

// ===== 5. ИНТЕРФЕЙСЫ =====
System.out.println("\n" + "=".repeat(70));
System.out.println("5. ДЕМОСТРАЦИЯ ИНТЕРФЕЙСОВ");
System.out.println("=".repeat(70));

// Массив интерфейсного типа Flyable
Flyable[] flyingObjects = new Flyable[] {
    boeing777,
    airbus380,
    boeing747F
};

System.out.println("Работа через интерфейс Flyable:");
for (Flyable flyable : flyingObjects) {
    System.out.println("\nFuel consumption: " +
flyable.getFuelConsumption() + " L/h");
}

// ===== 6. ПЕРЕГРУЗКА МЕТОДОВ =====
System.out.println("\n" + "=".repeat(70));
System.out.println("6. ДЕМОСТРАЦИЯ ПЕРЕГРУЗКИ МЕТОДОВ");
System.out.println("=".repeat(70));

System.out.println("\nПерегрузка метода fly():");
boeing777.takeOff();
boeing777.fly(); // fly() без параметров
boeing777.fly(12000); // fly(int altitude)

```

```

        boeing777.fly(11000, "North-East"); // fly(int altitude, String
direction)
        boeing777.land();

        System.out.println("\nПерегрузка метода boardPassengers():");
        airbus380.boardPassengers();           // без параметров
        airbus380.boardPassengers(850);        // с одним параметром
        airbus380.boardPassengers(100, 750);    // с двумя параметрами

        System.out.println("\nПерегрузка метода loadCargo():");
        boeing747F.loadCargo();                 // без параметров
        boeing747F.loadCargo(120000.0);         // с весом
        boeing747F.loadCargo(130000.0, "Electronics"); // с весом и типом

        // ===== 7. АБСТРАКТНЫЕ КЛАССЫ =====
        System.out.println("\n" + "=".repeat(70));
        System.out.println("7. ДЕМОНСТРАЦИЯ АБСТРАКТНЫХ КЛАССОВ");
        System.out.println("=".repeat(70));

        System.out.println("Vehicle - абстрактный класс с абстрактными
методами:");
        System.out.println("- abstract void start()");
        System.out.println("- abstract void stop()");
        System.out.println("- abstract String getType()");
        System.out.println("\nНельзя создать экземпляр: new Vehicle() - ошибка
компиляции");
        System.out.println("Подклассы обязаны реализовать абстрактные методы.");

        // ===== 8. ПОЛНЫЙ ЦИКЛ РАБОТЫ =====
        System.out.println("\n" + "=".repeat(70));
        System.out.println("8. ПОЛНЫЙ ЦИКЛ РАБОТЫ С ОБЪЕКТАМИ");
        System.out.println("=".repeat(70));

        System.out.println("\n--- Пассажирский рейс ---");
        airbus380.start();
        airbus380.takeOff();
        airbus380.fly(11000, "Paris");
        airbus380.serveMeal();
        airbus380.land();
        airbus380.stop();

        System.out.println("\n--- Грузовой рейс ---");
        boeing747F.start();
        boeing747F.loadCargo(135000.0, "Medical supplies");
        boeing747F.takeOff();
        boeing747F.fly(10500);
        boeing747F.land();
        boeing747F.unloadCargo();
        boeing747F.stop();

        // ===== 9. ОБСЛУЖИВАНИЕ (Интерфейс Maintainable) =====

```

```

System.out.println("\n" + "=".repeat(70));
System.out.println("9. ДЕМОНСТРАЦИЯ ИНТЕРФЕЙСА MAINTAINABLE");
System.out.println("=".repeat(70));

Maintainable[] maintenanceQueue = new Maintainable[] {
    boeing777,
    airbus380,
    boeing747F
};

System.out.println("\nПроверка необходимости обслуживания:");
for (int i = 0; i < maintenanceQueue.length; i++) {
    Maintainable item = maintenanceQueue[i];
    Aeroplane plane = (Aeroplane) item; // Приведение типа

    System.out.println("\n" + plane.getModel() + ":");
    System.out.println("Flight hours: " + plane.getFlightHours());
    System.out.println("Maintenance interval: " +
item.getMaintenanceInterval() + " hours");
    System.out.println("Needs maintenance: " + (item.needsMaintenance()
? "YES" : "NO"));

    if (item.needsMaintenance()) {
        item.performMaintenance();
    }
}

// ===== 10. ИНКАПСУЛЯЦИЯ =====
System.out.println("\n" + "=".repeat(70));
System.out.println("10. ДЕМОНСТРАЦИЯ ИНКАПСУЛЯЦИИ");
System.out.println("=".repeat(70));

System.out.println("\nВсе поля классов объявлены как private:");
System.out.println("- Прямой доступ к полям запрещен");
System.out.println("- Доступ осуществляется через публичные методы
(геттеры)");
System.out.println("\nПример:");
System.out.println("boeing777.getManufacturer() = " +
boeing777.getManufacturer());
System.out.println("boeing777.getMaxSpeed() = " +
boeing777.getMaxSpeed() + " km/h");
System.out.println("boeing777.getPassengerCapacity() = " +
boeing777.getPassengerCapacity());

// ===== ИТОГИ =====
System.out.println("\n" + "=".repeat(70));
System.out.println("ИТОГОВАЯ СВОДКА");
System.out.println("=".repeat(70));
System.out.println("✓ Конструкторы и их перегрузка - реализовано");
System.out.println("✓ Перегрузка методов - реализовано");

```

```

        System.out.println("✓ Наследование IS-A - реализовано (Vehicle →
Aeroplane → PassengerPlane/CargoPlane)");
        System.out.println("✓ Агрегация HAS-A - реализовано (Aeroplane содержит
Engine)");
        System.out.println("✓ Виртуальные методы - реализовано (displayInfo,
start, stop, getType)");
        System.out.println("✓ Абстрактные классы - реализовано (Vehicle)");
        System.out.println("✓ Полиморфизм - реализовано (массивы базового
типа)");
        System.out.println("✓ Интерфейсы - реализовано (Flyable,
Maintainable)");
        System.out.println("✓ Инкапсуляция - реализовано (private поля, public
методы)");
        System.out.println("=".repeat(70));
    }
}

```

4 Результат работы программы

```
=====
ДЕМОНСТРАЦИЯ ОБЪЕКТНОЙ МОДЕЛИ В JAVA - AEROPLANE
=====
```

```
=====
1. ДЕМОНСТРАЦИЯ ПЕРЕГРУЗКИ КОНСТРУКТОРОВ
=====
```

```
Конструктор без параметров: Engine: Unknown, HP: 1000, Consumption: 5.0 L/h
Конструктор с 1 параметром: Engine: Rolls-Royce Trent 900, HP: 1000, Consumption: 5.0 L/h
Конструктор с 2 параметрами: Engine: CFM56-7B, HP: 27300, Consumption: 5.0 L/h
Конструктор с 3 параметрами: Engine: GE90-115B, HP: 52000, Consumption: 12.5 L/h
```

```
=====
2. ДЕМОНСТРАЦИЯ HAS-A RELATIONSHIP (АГРЕГАЦИЯ)
=====
```

```
? Самолет Boeing 777 СОДЕРЖИТ (HAS-A) двигатель GE90-115B
```

```
=====
3. ДЕМОНСТРАЦИЯ IS-A RELATIONSHIP (НАСЛЕДОВАНИЕ)
=====
```

```
? PassengerPlane IS-A Aeroplane IS-A Vehicle
? CargoPlane IS-A Aeroplane IS-A Vehicle
```

```
=====
4. ДЕМОНСТРАЦИЯ ПОЛИМОРФИЗМА И ВИРТУАЛЬНЫХ МЕТОДОВ
=====
```

```
Полиморфное поведение - вызов методов через базовый тип:
```

```
--- Объект 1 ---
```

```
Vehicle: Boeing 777-300ER (2020)
Max Speed: 905.0 km/h
Passenger Capacity: 396
Wingspan: 64.8 m
Flight Hours: 0
Engine: GE90-115B, HP: 52000, Consumption: 12.5 L/h
Type: Aeroplane
```

```
--- Объект 2 ---
```

```
Vehicle: Airbus A380 (2021)
Max Speed: 945.0 km/h
Passenger Capacity: 853
Wingspan: 79.8 m
Flight Hours: 0
Engine: Rolls-Royce Trent 900, HP: 36300, Consumption: 11.8 L/h
Business Class Seats: 100
```

Economy Class Seats: 753

WiFi Available: Yes

Type: Passenger Plane

--- Объект 3 ---

Vehicle: Boeing 747-8F (2019)

Max Speed: 908.0 km/h

Passenger Capacity: 0

Wingspan: 68.4 m

Flight Hours: 0

Engine: GE CF6-80C2, HP: 28000, Consumption: 10.5 L/h

Max Cargo Weight: 140000.0 kg

Cargo Volume: 858.0 m³

Loading Ramp: Yes

Type: Cargo Plane

=====

5. ДЕМОНСТРАЦИЯ ИНТЕРФЕЙСОВ

=====

Работа через интерфейс Flyable:

Fuel consumption: 49.5 L/h

Fuel consumption: 100.65400000000001 L/h

Fuel consumption: 0.0 L/h

=====

6. ДЕМОНСТРАЦИЯ ПЕРЕГРУЗКИ МЕТОДОВ

=====

Перегрузка метода fly():

>>> 777-300ER is taking off...

Engine GE90-115B quick starting!

Accelerating on runway...

Lifting off!

Flying at 10000 meters altitude.

Flying at 12000 meters altitude.

Flying at 11000 meters altitude.

Heading: North-East

<<< 777-300ER is landing...

Descending...

Touchdown!

Taxiing to gate.

```
Перегрузка метода boardPassengers():
Boarding passengers...
Business class boarding first.
Economy class now boarding.
Boarding 850 passengers...
Boarding 100 business and 750 economy passengers.
```

```
Перегрузка метода loadCargo():
Loading cargo...
Loading 120000.0 kg of cargo.
Loading 130000.0 kg of cargo.
Cargo type: Electronics
```

=====

7. ДЕМОНСТРАЦИЯ АБСТРАКТНЫХ КЛАССОВ

=====

```
Vehicle - абстрактный класс с абстрактными методами:
- abstract void start()
- abstract void stop()
- abstract String getType()
```

Нельзя создать экземпляр: new Vehicle() - ошибка компиляции
Подклассы обязаны реализовать абстрактные методы.

=====

8. ПОЛНЫЙ ЦИКЛ РАБОТЫ С ОБЪЕКТАМИ

=====

--- Пассажирский рейс ---

```
=== Starting Airbus A380 ===
Engine Rolls-Royce Trent 900 starting...
Aircraft systems initialized.

>>> A380 is taking off...
Engine Rolls-Royce Trent 900 quick starting!
Accelerating on runway...
Lifting off!
Flying at 11000 meters altitude.
Heading: Paris
Serving meals to passengers.

<<< A380 is landing...
Descending...
Touchdown!
Taxiing to gate.
```


=== Stopping Airbus A380 ===
Engine Rolls-Royce Trent 900 stopped.
Aircraft systems shut down.

--- Грузовой рейс ---

=== Starting Boeing 747-8F ===
Engine GE CF6-80C2 starting...
Aircraft systems initialized.
Loading 135000.0 kg of cargo.
Cargo type: Medical supplies

>>> 747-8F is taking off...
Engine GE CF6-80C2 quick starting!
Accelerating on runway...
Lifting off!
Flying at 10500 meters altitude.

<<< 747-8F is landing...
Descending...
Touchdown!
Taxiing to gate.
Unloading cargo...

=== Stopping Boeing 747-8F ===
Engine GE CF6-80C2 stopped.
Aircraft systems shut down.

=====

9. ДЕМОНСТРАЦИЯ ИНТЕРФЕЙСА MAINTAINABLE

=====

Проверка необходимости обслуживания:

777-300ER:
Flight hours: 2
Maintenance interval: 100 hours
Needs maintenance: NO

A380:
Flight hours: 2
Maintenance interval: 100 hours
Needs maintenance: NO

747-8F:
Flight hours: 2
Maintenance interval: 100 hours

Needs maintenance: NO

10. ДЕМОНСТРАЦИЯ ИНКАПСУЛЯЦИИ

Все поля классов объявлены как `private`:

- Прямой доступ к полям запрещен
- Доступ осуществляется через публичные методы (геттеры)

Пример:

```
boeing777.getManufacturer() = Boeing
boeing777.getMaxSpeed() = 905.0 km/h
boeing777.getPassengerCapacity() = 396
```

ИТОГОВАЯ СВОДКА

- ? Конструкторы и их перегрузка - реализовано
- ? Перегрузка методов - реализовано
- ? Наследование IS-A - реализовано (Vehicle ? Aeroplane ? PassengerPlane/CargoPlane)
- ? Агрегация HAS-A - реализовано (Aeroplane содержит Engine)
- ? Виртуальные методы - реализовано (displayInfo, start, stop, getType)
- ? Абстрактные классы - реализовано (Vehicle)
- ? Полиморфизм - реализовано (массивы базового типа)
- ? Интерфейсы - реализовано (Flyable, Maintainable)
- ? Инкапсуляция - реализовано (private поля, public методы)

Выводы

В результате выполнения работы я приобрёл знания об объектной модели в языке программирования Java. Поскольку у меня уже был опыт работы с объектными моделями в других языках, выполнять задание было несложно — многие концепции оказались весьма схожими.

В рамках работы был разработан класс `Aeroplane`, который впоследствии был расширен за счёт создания нескольких новых классов. Также были реализованы различные интерфейсы, связанные с предметной областью «авиация».

В процессе работы были применены ключевые принципы ООП: наследование, инкапсуляция и полиморфизм. Кроме того, я изучил особенности ООП в Java — в частности, тот факт, что все методы, не помеченные как `final`, по умолчанию являются виртуальными.